

HO CHI MINH UNIVERSITY OF TECHNOLOGY
FACULTY OF IC DESIGN



Timer IP

Design and Verification

Final Project Report

Course: Foudation of IC design
Instructor: Mr. Kham Kha
Student: Nguyen Nhat Khanh

Ho Chi Minh City, September 24, 2026

Abstract

Contents

Abstract	i
List of Abbreviations	vi
1 Introduction, IP Specification and Waveforms	1
1.1 Introduction	1
1.1.1 Background and Motivation	1
1.1.2 Objectives and Scope	1
1.1.3 Design and Verification Flow	2
1.1.4 Report Organization	2
1.2 IP Specification	3
1.2.1 Interface Description	3
1.2.2 Register Map	3
1.3 Specification Waveforms	7
1.3.1 Register Write Transfer	7
1.3.2 Register Read Transfer	7
1.3.3 Counter Operation	8
2 RTL Design	9
2.1 Block Diagram	9
2.1.1 Block Descriptions	10
2.2 Logic Diagram	11
2.2.1 Counter Logic	11
2.2.2 Counter Logic	12

2.2.3	Counter Logic	13
2.2.4	Counter Logic	14
2.2.5	Counter Logic	15
2.2.6	Counter Logic	15
2.2.7	Counter Logic	16
2.3	RTL Source Files	17
2.3.1	Top-level Module — <code>timer_top.v</code>	17
2.3.2	Register File Module — <code>register.v</code>	17
2.3.3	Counter Control Module — <code>cnt_ctrl.v</code>	17
2.3.4	Counter Module — <code>cnt.v</code>	17
2.3.5	APB Slave Interface Module — <code>apb_slave.v</code>	17
2.3.6	Interrupt Module — <code>interrupt.v</code>	18
3	Verification	19
3.1	Verification Strategy and Environment	19
3.1.1	Verification Approach	19
3.2	Verification Plan (VPlan)	21
3.3	Testbench Organization	23
3.3.1	Top-level Testbench and Golden Reference Model	23
3.3.2	Bus Driver Tasks	23
3.3.3	Directed Test Sequences	24
3.4	Simulation Results	25
3.4.1	Test Results Summary	25
3.4.2	Coverage Results	26
A	Additional Material	27
A.1	Source Listings	27

List of Figures

1.1	Register Write Transfer (support 1 wait state)	7
1.2	Register Read Transfer (support 1 wait state)	7
1.3	Counter counting behavior across different prescaler settings	8
1.4	Counter operation in Halt Mode (div_val = 2)	8
1.5	Counter Interrupt (TCMP = 64'h4)	8
2.1	Block diagram of the Timer IP	9
2.2	Logic diagram of the abp	11
2.3	Logic diagram of the register0	12
2.4	Logic diagram of the register1	13
2.5	Logic diagram of the register2	14
2.6	Logic diagram of the cnt_ctrl	15
2.7	Logic diagram of the Interrupt	15
2.8	Logic diagram of the cnt	16
3.1	Vplan1	21
3.2	Vplan2	21
3.3	Vplan3	22
3.4	Vplan4	22
3.5	Vplan5	23
3.6	result	25
3.7	Coverage1	26
3.8	Coverage2	26

List of Tables

1.1	Port list of the Timer IP	3
1.2	Register map summary	3
1.3	Timer Control Register (TCR) Field Description	4
1.4	Timer Data Register 0 (TDR0) Field Description	5
1.5	Timer Data Register 1 (TDR1) Field Description	5
1.6	Timer Compare Register 0 (TCMP0) Field Description	5
1.7	Timer Compare Register 1 (TCMP1) Field Description	5
1.8	Timer Interrupt Enable Register (TIER) Field Description	6
1.9	Timer Interrupt Status Register (TISR) Field Description	6
1.10	Timer Halt Control Status Register (THCSR) Field Description	6

List of Abbreviations

IP	Intellectual Property
RTL	Register Transfer Level
DUT	Design Under Test
TB	Testbench

Chapter 1

Introduction, IP Specification and Waveforms

1.1 Introduction

1.1.1 Background and Motivation

Timers are indispensable hardware blocks in modern Systems-on-Chip and microcontrollers. They provide accurate time intervals for software delays, periodic tick interrupts for Real-Time Operating Systems, and system event tracking. In processor architectures such as RISC-V, timer modules (like the CLINT) handle wall-clock time keeping and thread scheduling. Designing a standard AMBA APB-compliant Timer IP offers practical insights into bus interfacing, configurable clock prescalers, hardware interrupt handling, and industrial design verification flows.

1.1.2 Objectives and Scope

In Scope:

- Design of an APB Slave 64-bit count-up Timer IP supporting byte strobe (`tim_pstrb`), 1-cycle wait state (`tim_pready`), error reporting (`tim_pslverr`), and debug halt mode.
- Implementation of an 8-stage prescaler, 64-bit compare match, and a maskable level interrupt.
- Verification using a layered testbench, Golden Model checking, and achieving 100% code coverage (Statement, Branch, Condition, FEC Expression, Toggle).

Out of Scope: Physical layout (synthesis, place-and-route), multi-channel PWM outputs, and input capture modes.

1.1.3 Design and Verification Flow

The project follows the standard digital frontend flow:

1. **Specification Analysis:** Define IP functional requirements and register maps.
2. **RTL Design:** Implement synthesizable logic using Verilog/SystemVerilog.
3. **Verification Planning:** Formulate the test plan with corner-case scenarios.
4. **Testbench Development:** Construct bus drivers, monitors, and scoreboards.
5. **Simulation & Waveform Debug:** Run simulations and debug functional bugs via waveforms.
6. **Coverage Closure:** Refine stimulus iteratively to reach 100% code coverage.

Tools and Languages: Verilog; QuestaSim/ModelSim (simulator, waveform viewer, and coverage analysis engine); Make/Bash scripts for automated testing.

1.1.4 Report Organization

This report consists of four main chapters. Chapter 1 introduces the project background, design flow, and detailed IP specifications. Chapter 2 describes the micro-architecture, sub-module decomposition, and RTL logic diagrams. Chapter 3 presents the verification strategy, testbench components, and test plan. Finally, Chapter 4 summarizes the simulation results, bug tracking, and code coverage closure.

1.2 IP Specification

1.2.1 Interface Description

Table 1.1: Port list of the Timer IP

Port	Dir.	Width	Description
sys_clk	Input	1	System clock
sys_rst_n	Input	1	Active-low asynchronous reset
tim_psel	Input	1	APB slave select
tim_pwrite	Input	1	APB transfer direction (1: Write, 0: Read)
tim_penable	Input	1	APB enable strobe (access phase)
tim_paddr[11:0]	Input	12	APB register address bus
tim_pwdata[31:0]	Input	32	APB write data bus
tim_prdata[31:0]	Output	32	APB read data bus
tim_pstrb[3:0]	Input	4	APB byte-write strobe signals
tim_pready	Output	1	APB transfer ready handshake
tim_pslverr	Output	1	APB slave error response
tim_int	Output	1	Active-high level timer interrupt output
dbg_mode	Input	1	Debug mode status input from system

1.2.2 Register Map

Table 1.2: Register map summary

Offset	Name	Width	Access	Description
0x00	TCR	32	RW	Timer Control Register
0x04	TDR0	32	RW	Timer Data Register 0
0x08	TDR1	32	RW	Timer Data Register 1
0x0C	TCMP0	32	RW	Timer Compare Register 0
0x10	TCMP1	32	RW	Timer Compare Register 1
0x14	TIER	32	RW	Timer Interrupt Enable Register
0x18	TISR	32	RW1C	Timer Interrupt Status Register
0x1C	THCSR	32	Mixed	Timer Halt Control Status Register
Others	Reserved	32	RAZ/WI	Unmapped space

Register Bit-field Description

Table 1.3: Timer Control Register (TCR) Field Description

Bit	Name	Type	Reset	Description
31:12	Reserved	RO	20'h0	Reserved bits, read as zero.
11:8	div_val	RW	4'b0001	Prescaler divisor setting: • 4'b0000: Divide by 1 • 4'b0001: Divide by 2 (default) • 4'b0010: Divide by 4 • 4'b0011: Divide by 8 • 4'b0100: Divide by 16 • 4'b0101: Divide by 32 • 4'b0110: Divide by 64 • 4'b0111: Divide by 128 • 4'b1000: Divide by 256 • Others: Prohibited; generates slave error <i>Note:</i> Modification while <code>timer_en</code> = 1 is prohibited and triggers an error response
7:2	Reserved	RO	6'b0	Reserved bits.
1	div_en	RW	1'b0	Counter control mode enable: • 0: Disabled, counts with system clock • 1: Enabled, counting speed controlled by <code>div_val</code> <i>Note:</i> Modification while <code>timer_en</code> = 1 is prohibited and triggers an error response.
0	timer_en	RW	1'b0	Timer enable: • 0: Disabled, counter stops • 1: Enabled, counter starts counting <i>Note:</i> Transition from High to Low clears TDR0 and TDR1 to initial values.

Table 1.4: Timer Data Register 0 (TDR0) Field Description

Bit	Name	Type	Reset	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32-bit of 64-bit counter. <i>Note:</i> Cleared to initial value when <code>timer_en</code> transitions from High to Low.

Table 1.5: Timer Data Register 1 (TDR1) Field Description

Bit	Name	Type	Reset	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32-bit of 64-bit counter. <i>Note:</i> Cleared to initial value when <code>timer_en</code> transitions from High to Low.

Table 1.6: Timer Compare Register 0 (TCMP0) Field Description

Bit	Name	Type	Reset	Description
31:0	TCMP0	RW	32'hFFFFFF_FFFF	Lower 32-bit of 64-bit compare value. Interrupt pending bit is asserted when the counter value equals the compare value.

Table 1.7: Timer Compare Register 1 (TCMP1) Field Description

Bit	Name	Type	Reset	Description
31:0	TCMP1	RW	32'hFFFFFF_FFFF	Upper 32-bit of 64-bit compare value. Interrupt pending bit is asserted when the counter value equals the compare value.

Table 1.8: Timer Interrupt Enable Register (TIER) Field Description

Bit	Name	Type	Reset	Description
31:1	Reserved	RO	31'h0	Reserved bits, read as zero.
0	int_en	RW	1'b0	Timer interrupt enable: • 0: Disabled (no timer interrupt is asserted). • 1: Enabled (allows interrupt pending status to drive the output). <i>Note:</i> Clearing this bit to 0 while an interrupt is active masks <code>tim_int</code> to 0 immediately without clearing the pending status bit (<code>TISR.int_st</code>).

Table 1.9: Timer Interrupt Status Register (TISR) Field Description

Bit	Name	Type	Reset	Description
31:1	Reserved	RO	31'h0	Reserved bits, read as zero.
0	int_st	RW1C	1'b0	Timer interrupt trigger status (pending bit): • 0: Interrupt condition has not occurred. • 1: Interrupt condition occurred (counter equals compare value). • Write 1 to clear to 0; writing 0 has no effect. <i>Note 1:</i> Counter continues counting normally upon match. <i>Note 2:</i> Clear action has highest priority.

Table 1.10: Timer Halt Control Status Register (THCSR) Field Description

Bit	Name	Type	Reset	Description
31:2	Reserved	RO	30'h0	Reserved bits, read as zero.
1	halt_ack	RO	1'b0	Timer halt acknowledge: • 0: Timer is not halted. • 1: Timer is halted. <i>Note:</i> The timer accepts the halt request only in debug mode (<code>dbg_mode = 1</code>).
0	halt_req	RW	1'b0	Timer halt request: • 0: No halt request. • 1: Request timer to halt.

1.3 Specification Waveforms

1.3.1 Register Write Transfer

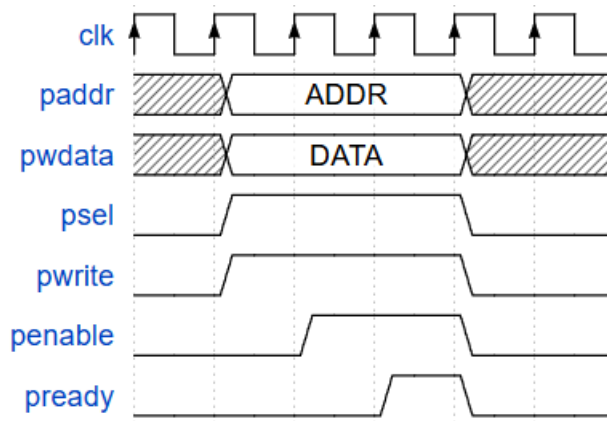


Figure 1.1: Register Write Transfer (support 1 wait state)

1.3.2 Register Read Transfer

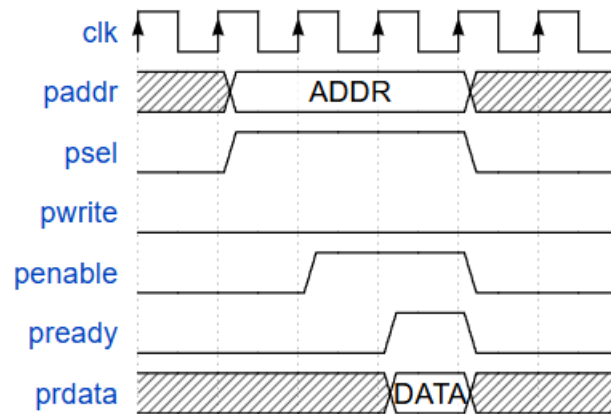


Figure 1.2: Register Read Transfer (support 1 wait state)

1.3.3 Counter Operation

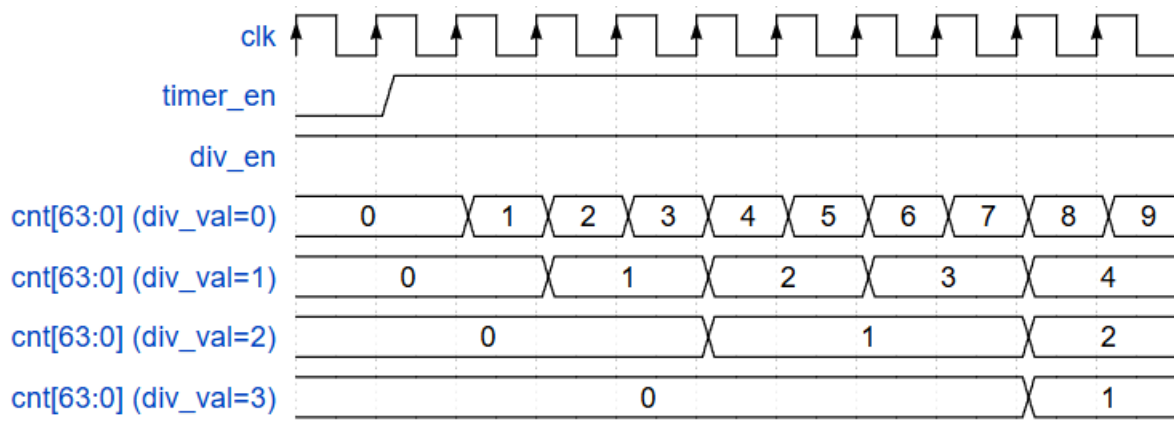


Figure 1.3: Counter counting behavior across different prescaler settings

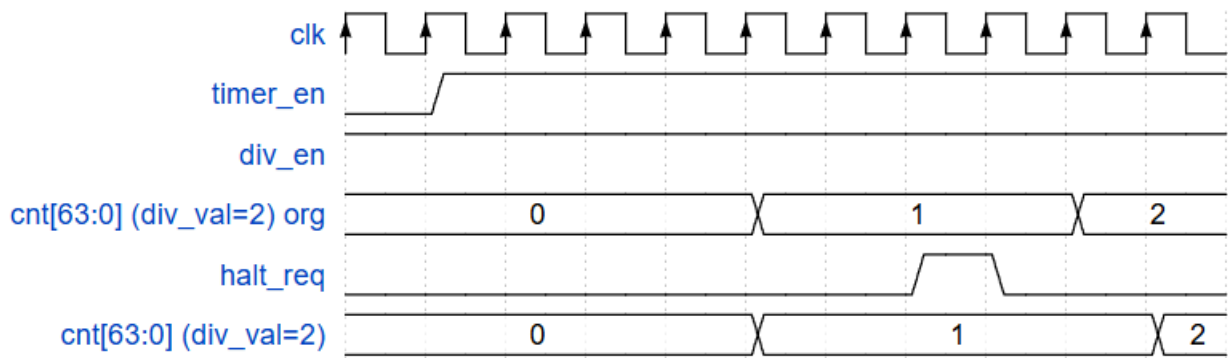


Figure 1.4: Counter operation in Halt Mode (div_val = 2)

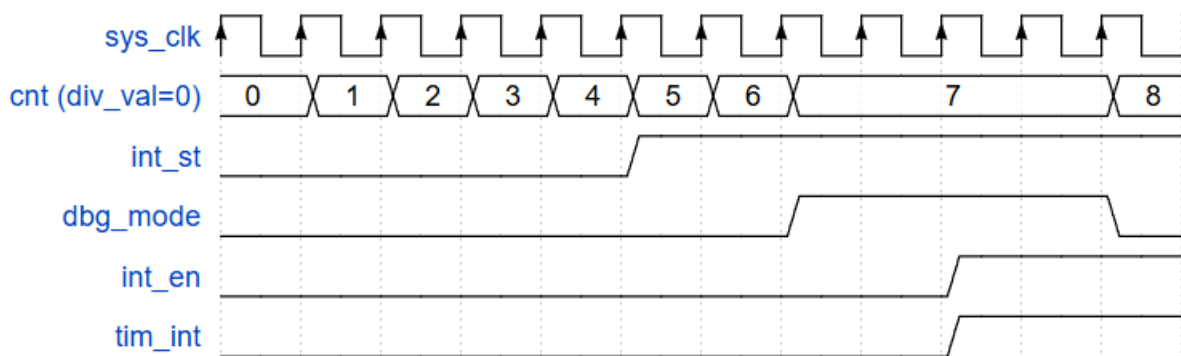


Figure 1.5: Counter Interrupt (TCMP = 64'h4)

Chapter 2

RTL Design

2.1 Block Diagram

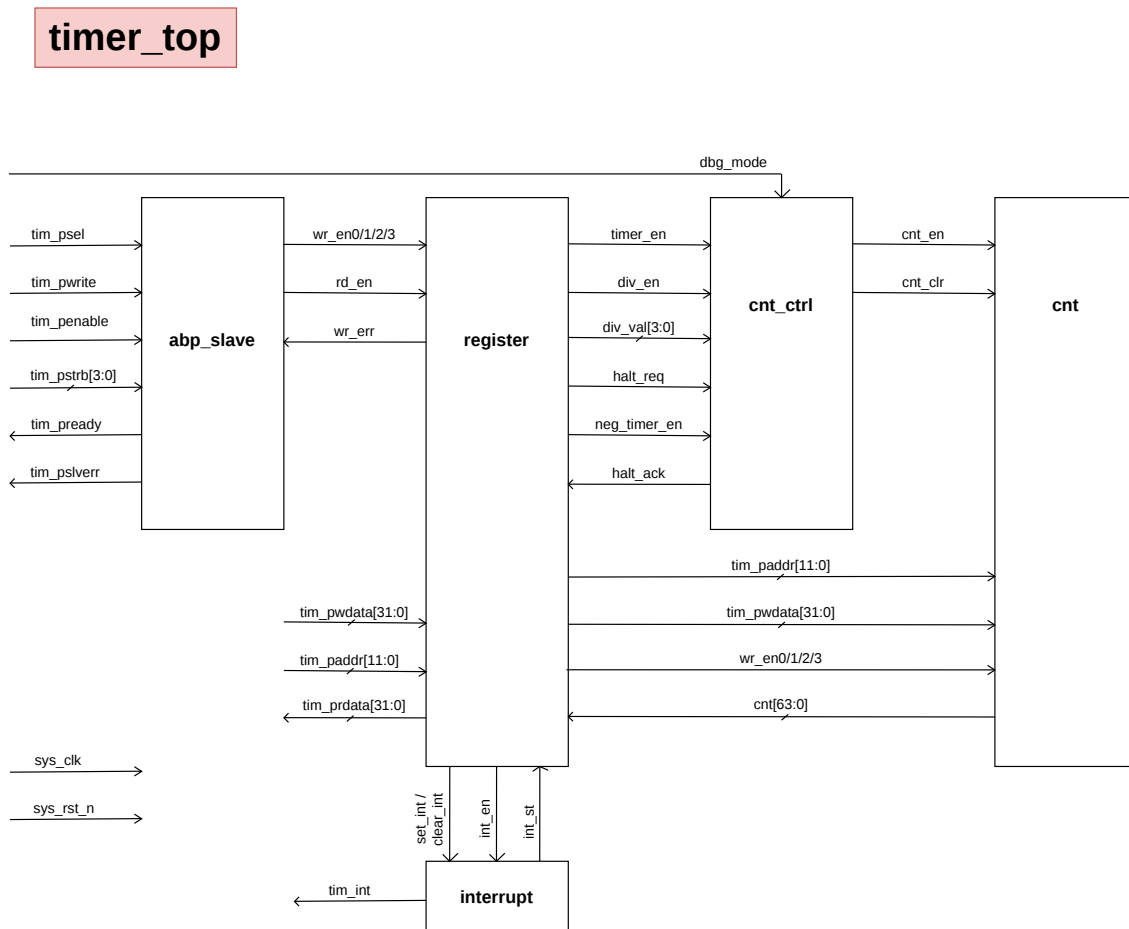


Figure 2.1: Block diagram of the Timer IP

2.1.1 Block Descriptions

APB Slave Interface (`abp_slave`): This module interfaces directly with the system AMBA APB bus, translating bus transactions into internal control signals. It samples `tim_psel`, `tim_pwrite`, and `tim_penable` to orchestrate transfer phases, generating individual byte-write strobes (`wr_en0/1/2/3`) from `tim_pstrb[3:0]` and a read-enable pulse (`rd_en`). Additionally, it inserts a 1-cycle wait state via `tim_pready` to optimize timing margins and asserts `tim_pslverr` whenever an illegal register access is flagged by `wr_err`.

Register File (`register`): This block implements the memory-mapped configuration and status registers, including TCR, TCMP0/1, TIER, TISR, and THCSR. It performs address decoding on `tim_paddr[11:0]`, controls read-data multiplexing to drive `tim_prdata[31:0]`, and enforces hardware access rules such as flagging prohibited configuration updates with `wr_err`. Furthermore, it distributes control signals (`timer_en`, `div_en`, `div_val`, `halt_req`) to downstream logic and detects the falling edge of the timer enable signal (`neg_timer_en`).

Counter Controller (`cnt_ctrl`): The counter controller regulates the counting speed and operational modes of the 64-bit counter. It integrates prescaler logic to generate periodic count-enable pulses (`cnt_en`) based on `div_val[3:0]` when `div_en` is active. In debug scenarios where `dbg_mode` is high, it halts timer progression upon receiving `halt_req` and responds with `halt_ack`. It also asserts `cnt_clr` upon detecting `neg_timer_en` to reset the counter registers back to zero in compliance with advanced requirements.

64-bit Counter Datapath (`cnt`): This block forms the core datapath containing the 64-bit count-up registers, physically divided into TDR0 (lower 32 bits) and TDR1 (upper 32 bits). It increments sequentially on each `cnt_en` tick and performs synchronous clearance to zero when `cnt_clr` is asserted. The block supports direct bus writes via `tim_pwdata[31:0]` using byte-enable signals (`wr_en0/1/2/3`) and continuously broadcasts the current 64-bit count value (`cnt[63:0]`) to both the register file and the interrupt module.

Interrupt Module (`interrupt`): This module monitors compare-match conditions and manages interrupt generation. When the 64-bit count matches the programmed compare value (TCMP1:TCMP0), it sets the interrupt pending flag (`int_st`) via `set_int`. It supports Write-1-to-Clear (RW1C) clearing through `clear_int` and drives the active-high level interrupt line (`tim_int`), which is masked dynamically by the interrupt enable bit (`int_en`).

2.2 Logic Diagram

2.2.1 Counter Logic

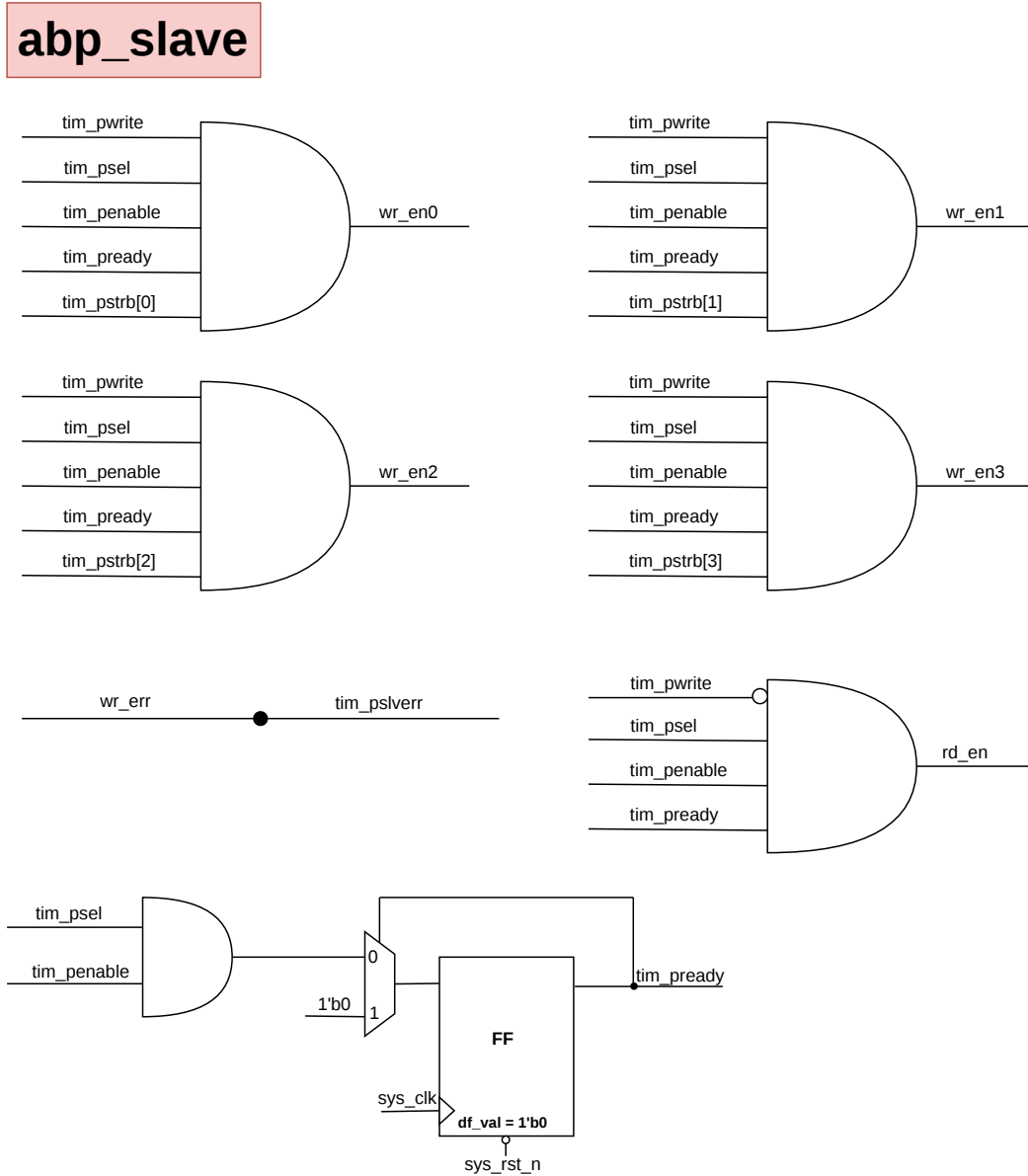


Figure 2.2: Logic diagram of the abp

2.2.2 Counter Logic

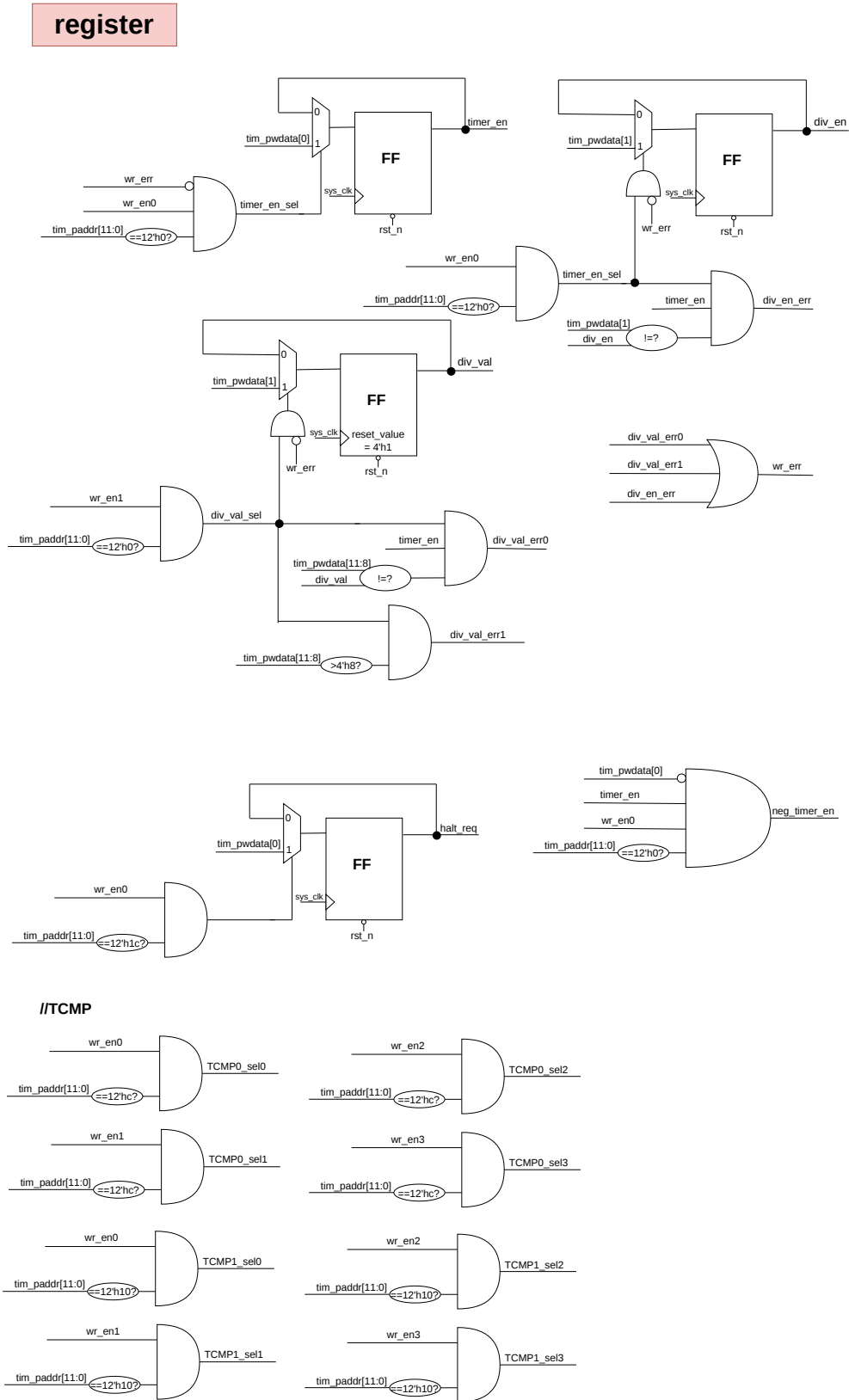


Figure 2.3: Logic diagram of the register0

2.2.3 Counter Logic

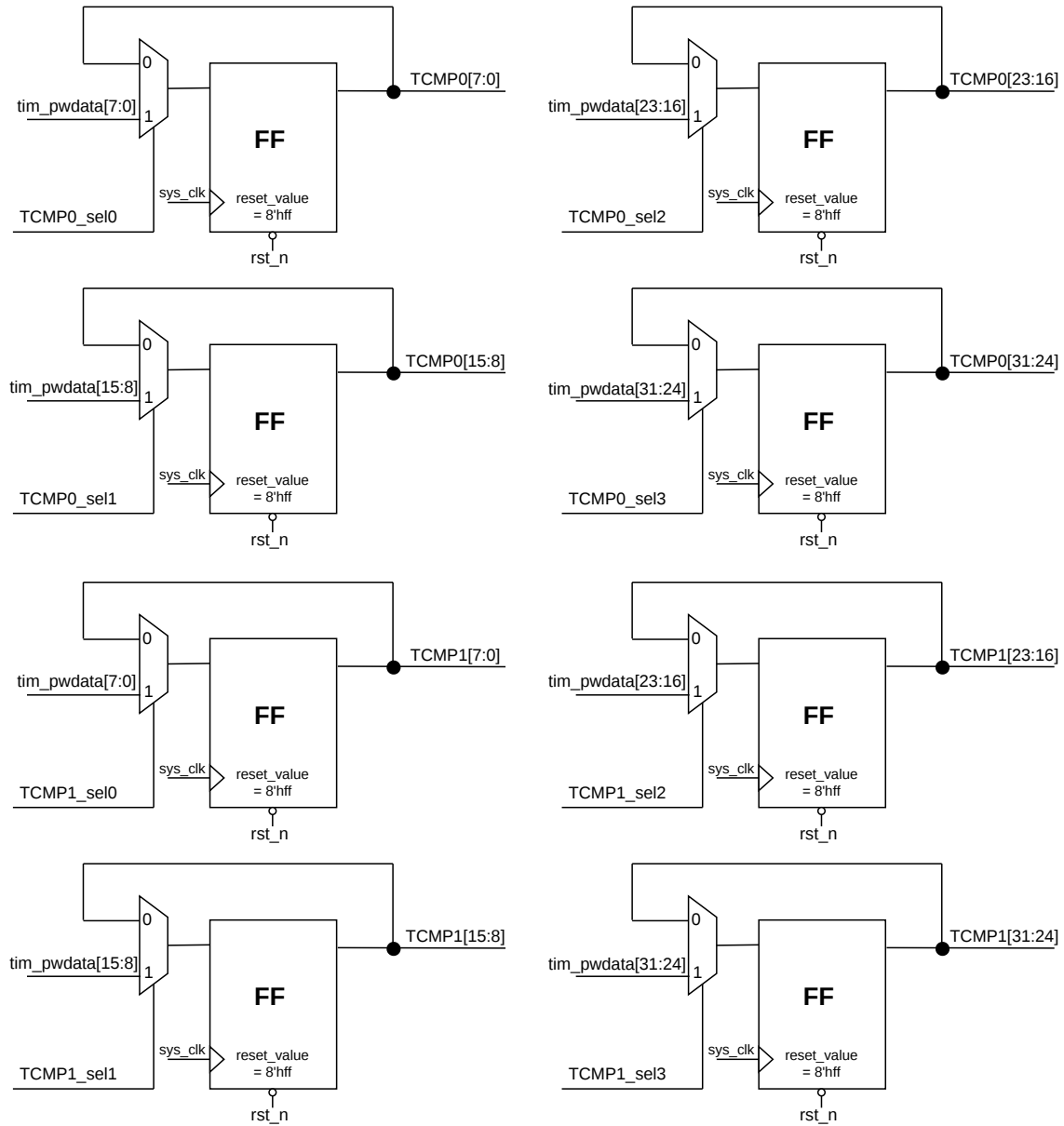


Figure 2.4: Logic diagram of the register1

2.2.4 Counter Logic

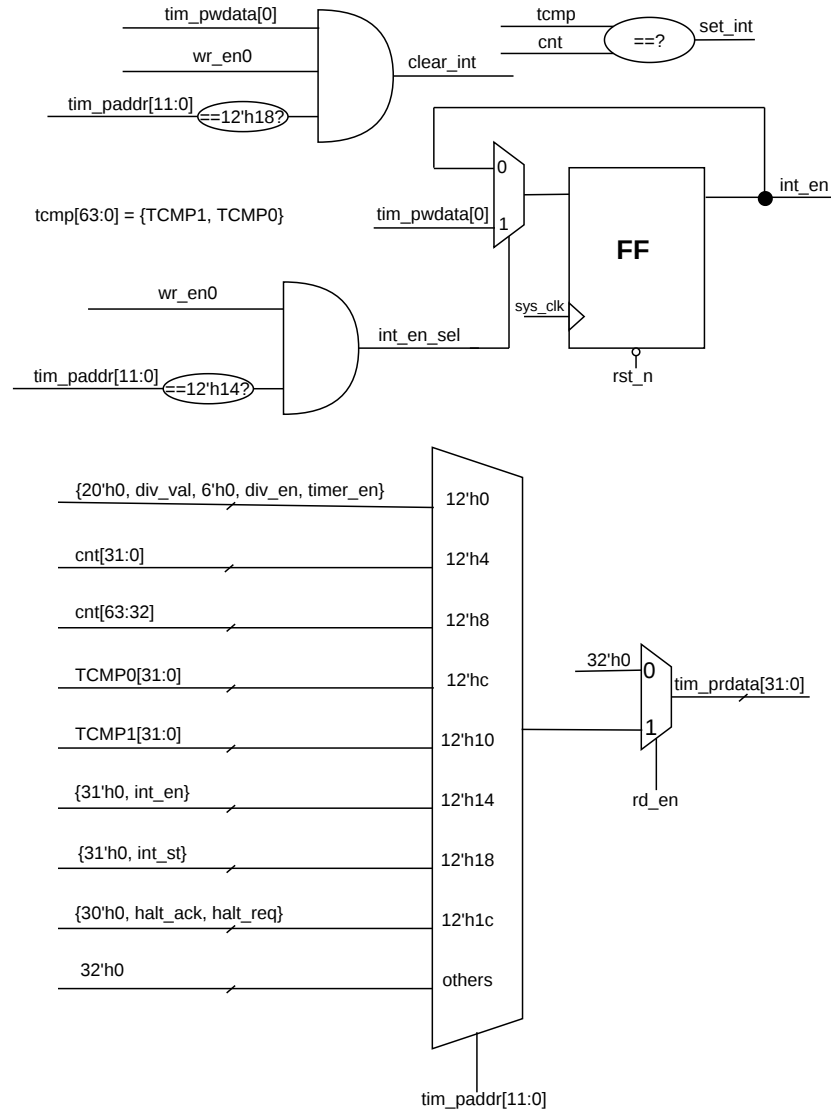
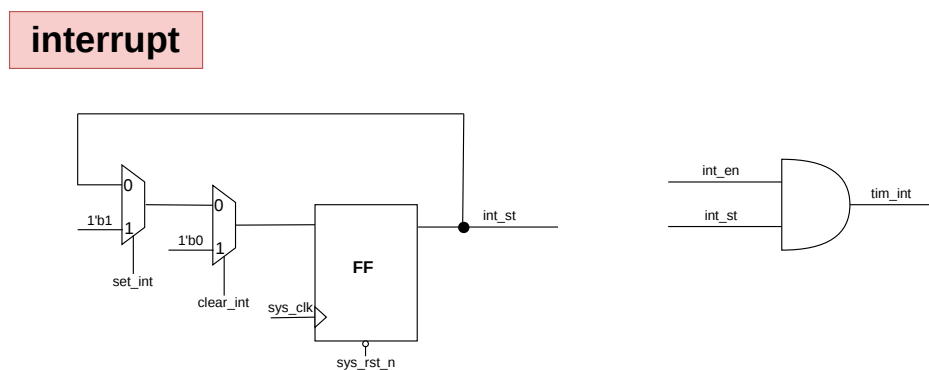
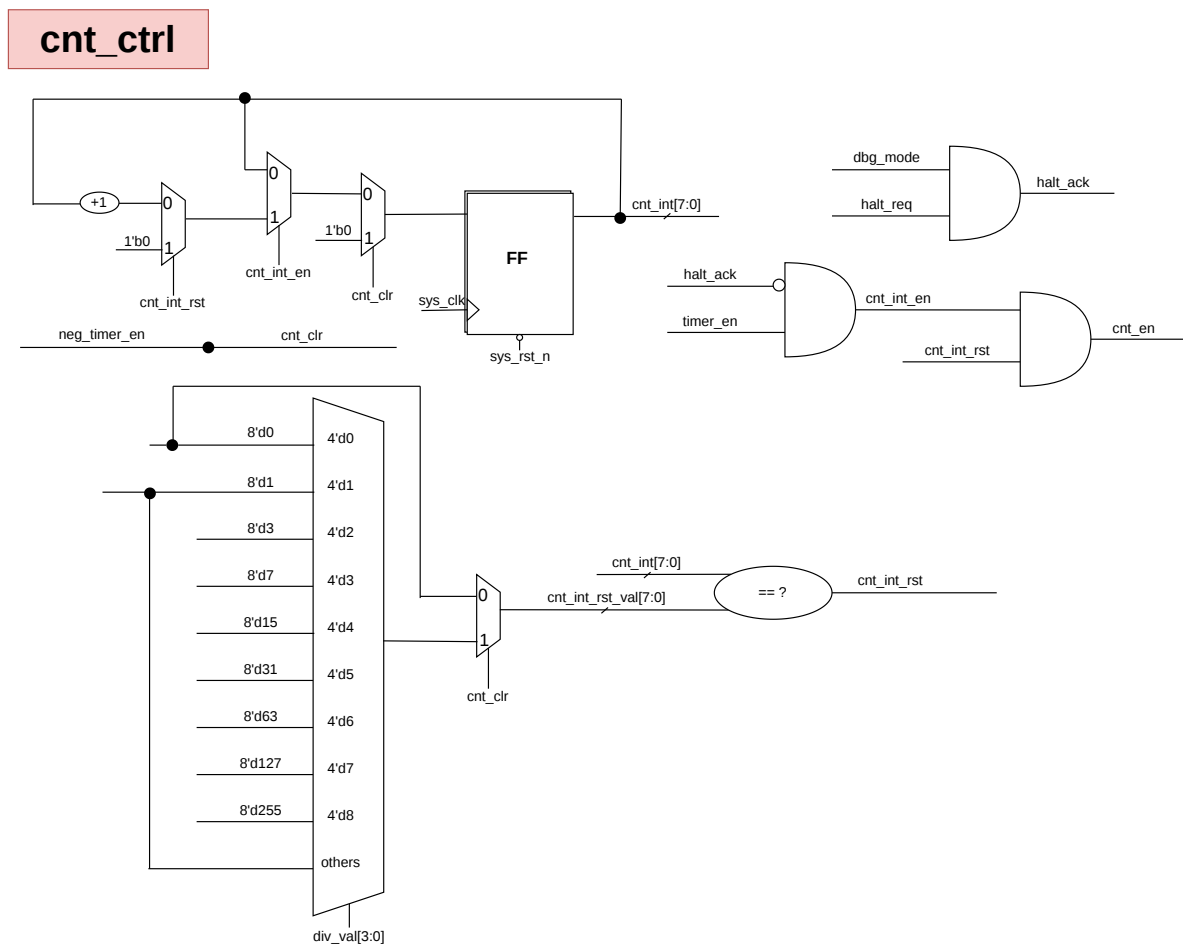


Figure 2.5: Logic diagram of the register2



2.2.7 Counter Logic

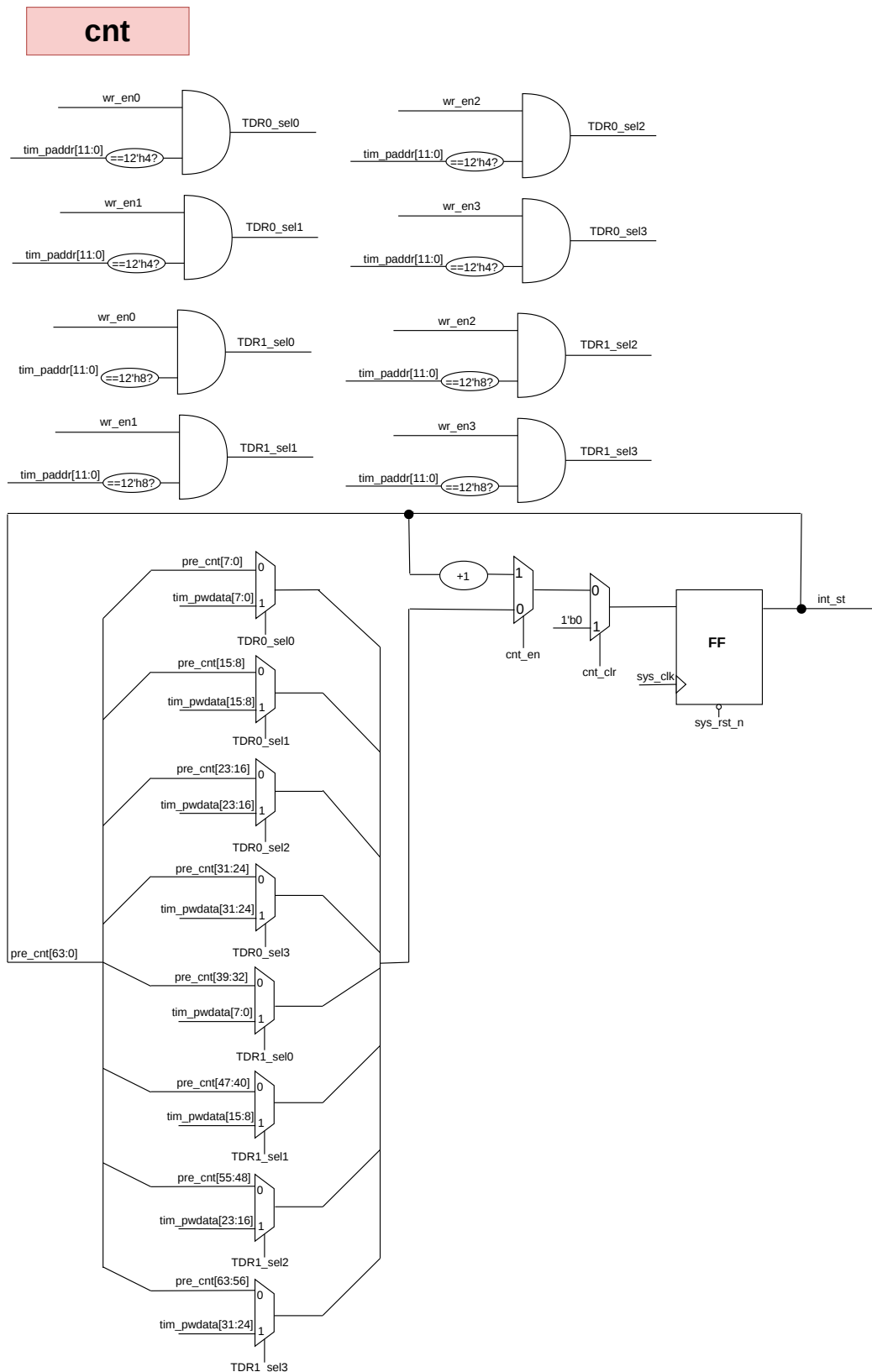


Figure 2.8: Logic diagram of the cnt

2.3 RTL Source Files

The synthesizable RTL implementing the architecture of Section 2.1 is organized as five sub-modules integrated by the top-level module, exactly matching the six Verilog files under `rtl/` in the project repository. The full source of every file below is available there; this section summarizes what each file implements.

2.3.1 Top-level Module — `timer_top.v`

Instantiates and interconnects the five sub-modules (`apb_slave`, `register`, `cnt_ctrl`, `cnt`, `interrupt`), routing the APB interface signals, the prescaler/control signals (`timer_en`, `div_en`, `div_val`, `halt_req`/`halt_ack`), the internal 64-bit counter bus (`cnt[63:0]`), and the interrupt set/clear signals between them.

2.3.2 Register File Module — `register.v`

Implements the memory-mapped configuration and status registers (TCR, TCMP0, TCMP1, TIER, THCSR, and the TISR read path). Decodes `tim_paddr[11:0]`, checks for prohibited configuration writes (changing `div_en`/`div_val` while `timer_en` = 1, or an out-of-range `div_val`) to generate `wr_err`, multiplexes read data onto `tim_prdata[31:0]`, and detects the falling edge of `timer_en` (`neg_timer_en`).

2.3.3 Counter Control Module — `cnt_ctrl.v`

Generates the prescaler tick-enable pulse `cnt_en` from `div_val[3:0]` and `div_en`, computes the debug halt acknowledge `halt_ack` = `dbg_mode && halt_req`, and asserts the synchronous clear `cnt_clr` on `neg_timer_en`.

2.3.4 Counter Module — `cnt.v`

Holds the 64-bit up-counter (`cnt[63:0]`), physically split into TDR0 (lower 32 bits) and TDR1 (upper 32 bits). Increments on `cnt_en`, clears synchronously on `cnt_clr`, and accepts byte-masked APB writes to TDR0/TDR1 when the counter is not actively ticking.

2.3.5 APB Slave Interface Module — `apb_slave.v`

Generates the byte-write enables `wr_en0/1/2/3` and the read enable `rd_en` from `tim_pstrb[3:0]` and the APB handshake, drives the single-cycle wait state on `tim_pready`, and passes

through the slave error `tim_pslverr` from `wr_err`.

2.3.6 Interrupt Module — `interrupt.v`

Implements the Write-1-to-Clear interrupt-pending flip-flop `int_st` (set by `set_int`, cleared by `clear_int`, with the clear action taking priority), and drives the masked interrupt output `tim_int = int_en && int_st`.

Chapter 3

Verification

3.1 Verification Strategy and Environment

3.1.1 Verification Approach

The verification strategy adopts a simulation-based, self-checking layered testbench architecture to thoroughly validate the Timer IP against its functional specification and achieve 100% code coverage closure.

Self-Checking Architecture and Expected Value Generation

To ensure automated and deterministic checking, the testbench incorporates a cycle-accurate Reference Model (Golden Model) running in parallel with the DUT:

- **Reference Prediction:** The reference model maintains shadow register states and simulates the prescaler counter behavior to generate expected values (`exp_prdata`, `exp_pslverr`, and `exp_tim_int`).
- **Automated Scoreboard Comparison:** The scoreboard compares the APB response signals (`tim_prdata`, `tim_pready`, `tim_pslverr`) at the end of every transfer, and the interrupt output (`tim_int`) on every clock cycle, against the corresponding expected values:

`Match = (tim_prdata = exp_prdata), (tim_pslverr = exp_pslverr), (tim_int = exp_tim_int)`

Any discrepancy immediately increments an error counter and flags an assertion failure with the simulation timestamp and signal state.

Pin-Level and Interface Protocol Checking

The testbench continuously monitors and verifies the DUT interface pins:

- **APB Protocol Compliance:** Monitors the handshake sequence among `tim_psel`, `tim_penable`, and `tim_pready`. Verifies that `tim_pready` correctly inserts a 1-cycle wait state during access phases.
- **Error Response Handshake:** Checks that `tim_pslverr` asserts when prohibited operations occur (e.g., reconfiguring `div_en/div_val` while `timer_en = 1`, or writing out-of-range divisor values).
- **Byte-Strobe Write Verification:** Validates that byte-write strobes (`tim_pstrb[3:0]`) exclusively update the targeted byte lanes in registers without corrupting adjacent bytes.
- **Interrupt Output Check:** Confirms that `tim_int` asserts to High exactly when the 64-bit counter matches the compare value (`cnt == TCMP`) with `TIER.int_en = 1`, stays asserted until cleared via Write-1-to-Clear (`TISR.int_st`), and remains masked when `int_en = 0`.
- **Debug Mode and Halt Verification:** Asserts `dbg_mode` and sets `THCSR.halt_req`, verifying that `THCSR.halt_ack` is asserted and the counter freezes cleanly, maintaining correct count periods upon resuming.

100% Code Coverage Closure Strategy

To guarantee that 100% code coverage is met across all mandated metrics prior to sign-off, targeted test scenarios are developed:

- **Statement & Branch Coverage:** Directed testcases cover every operational branch, including all `div_val` configurations (1, 2, 4, ..., 256), hardware clear on `timer_en H → L`, and all error condition branches.
- **Condition & Focused Expression Coverage (FEC):** Ensures all independent terms in boolean logic (such as write-enable logic, prescaler rollover expressions, and comparator equality) are exercised in both masking and non-masking states to achieve 100% FEC.
- **Toggle Coverage:** Verifies that every single bit of the 12-bit address bus, 32-bit read/write data buses, strobe signals, and internal 64-bit counter transitions through both $0 \rightarrow 1$ and $1 \rightarrow 0$ states.

3.2 Verification Plan (VPlan)

ID	Item	Description	Test sequence	Pass condition
1.1	reg_init_chk	Verify TCR resets to its documented default value.	Apply sys_rst_n=0 then release it (no APB access performed). Read TCR (addr=0x000).	tim_prdata = 32'h0000_0100 (div_val=4'h1, div_en=0, timer_en=0).
1.2		Verify TDR0 resets to its documented default value.	After reset, read TDR0 (addr=0x004).	tim_prdata = 32'h0000_0000.
1.3		Verify TDR1 resets to its documented default value.	After reset, read TDR1 (addr=0x008).	tim_prdata = 32'h0000_0000.
1.4		Verify TCMP0 resets to its documented default value.	After reset, read TCMP0 (addr=0x00C).	tim_prdata = 32'hFFFF_FFFF.
1.5		Verify TCMP1 resets to its documented default value.	After reset, read TCMP1 (addr=0x010).	tim_prdata = 32'hFFFF_FFFF.
1.6		Verify TIER resets to its documented default value.	After reset, read TIER (addr=0x014).	tim_prdata = 32'h0000_0000.
1.7		Verify TISR resets to its documented default value.	After reset, read TISR (addr=0x018).	tim_prdata = 32'h0000_0000.
1.8		Verify THCSR resets to its documented default value.	After reset, read THCSR (addr=0x01C).	tim_prdata = 32'h0000_0000.
2.1	reg_rw_chk	Verify TDR0 is fully read/write capable with full-word access.	With timer_en=0, write 0x0000_0000, 0xFFFF_FFFF, 0x5555_5555, 0xAAAA_AAAA to TDR0 (pstrb=4'hF); read back after each write.	Each readback equals the value just written.
2.2		Verify TDR1 is fully read/write capable with full-word access.	Same 4-pattern write/readback sequence applied to TDR1 (addr=0x008).	Each readback equals the value just written.
2.3		Verify TCMP0 is fully read/write capable.	Same 4-pattern write/readback sequence applied to TCMP0 (addr=0x00C).	Each readback equals the value just written.
2.4		Verify TCMP1 is fully read/write capable.	Same 4-pattern write/readback sequence applied to TCMP1 (addr=0x010).	Each readback equals the value just written.
2.5		Verify TCR.div_val field is writable across its full legal range.	With timer_en=0, write TCR.div_val = 0,1,2,...,8 one at a time; read back TCR after each write.	TCR[11:8] readback equals the value written each time; tim_pslverr=0 for every write.
2.6		Verify TIER.int_en bit is read/write while remaining bits stay read-only 0.	Write TIER=0xFFFF_FFFF, read back; then write TIER=0x0, read back.	Readback = 0x0000_0001 after the first write, 0x0000_0000 after the second (only bit0 follows pwdata).
2.7		Verify THCSR.halt_req bit is read/write independent of halt_ack.	With dbg_mode=0 (halt_ack forced 0), write THCSR bit0=1 then 0; read back after each write.	THCSR[0] readback follows the write each time; THCSR[1] (halt_ack) reads 0 throughout.

Figure 3.1: Vplan1

ID	Item	Description	Test sequence	Pass condition
3.1	reg_reserved_chk	Verify unmapped addresses are Read-As-Zero / Write-Ignored (RAZ/WI).	Write a non-zero pattern (e.g. 0xFFFF_FFFF) to an unmapped address (e.g. 0x020, 0x100, 0xFFF); read the same address; read another unmapped address without any prior write.	All reads from unmapped addresses return 32'h0000_0000; no defined register is affected by the write.
3.2		Verify reserved bit-fields inside valid registers are RAZ/WI.	Write 0xFFFF_FFFF (full word) to TCR, TIER, TISR and THCSR; read back each register.	Only the documented live bits reflect the write (TCR[11:8,1:0], TIER[0], TISR[0] per RW1C rule, THCSR[0]); every reserved bit position reads back 0.
4.1	reg_byte_access	Verify byte-lane 0 write isolation.	Preload TDR0=0xAAAA_AAAA (timer_en=0); write only byte0 (pstrb=4'b0001) with pwdata[7:0]=0x11; read back.	TDR0 = 0xAAAA_AA11 (only bits[7:0] changed).
4.2		Verify byte-lane 1 write isolation.	Preload TDR0=0xAAAA_AAAA; write only byte1 (pstrb=4'b0010) with pwdata[15:8]=0x22; read back.	TDR0 = 0xAAAA_22AA (only bits[15:8] changed).
4.3		Verify byte-lane 2 write isolation.	Preload TDR0=0xAAAA_AAAA; write only byte2 (pstrb=4'b0100) with pwdata[23:16]=0x33; read back.	TDR0 = 0xAA33_AAAA (only bits[23:16] changed).
4.4		Verify byte-lane 3 write isolation.	Preload TDR0=0xAAAA_AAAA; write only byte3 (pstrb=4'b1000) with pwdata[31:24]=0x44; read back.	TDR0 = 0x44AA_AAAA (only bits[31:24] changed).
4.5		Verify simultaneous, non-contiguous byte-lane write.	Preload TCMP0=0x0000_0000; single write with pstrb=4'b1010 (byte1 and byte3), pwdata=0xB00DD00.	TCMP0 = 0xBB00_DD00 (only byte1 and byte3 updated; byte0/byte2 remain 0x00).
4.6		Verify a write with no byte lane selected has no effect.	Preload TCMP1=0x1234_5678; issue a write with pstrb=4'b0000 and arbitrary pwdata.	TCMP1 readback remains 0x1234_5678, unchanged.
5.1	cnt_ctrl_chk	Verify div_val can be changed while the counter is stopped.	With timer_en=0, write div_val=5; read back TCR.	TCR[11:8]=5; tim_pslverr=0 for this write.
5.2		Verify a prohibited div_val value is rejected.	With timer_en=0, write div_val=4'hF (>4'h8); read back TCR.	tim_pslverr asserted during the write; TCR[11:8] retains its previous value (unchanged).
5.3		Verify div_val cannot be changed while the counter is running.	Set timer_en=1 with div_val=2; attempt to write div_val=4 while timer_en stays 1; read back TCR.	tim_pslverr asserted during the write; TCR[11:8] remains 2 (unchanged).
5.4		Verify div_en cannot be changed while the counter is running.	Set timer_en=1 with div_en=0; attempt to write div_en=1 while timer_en stays 1; read back TCR.	tim_pslverr asserted during the write; TCR[1] remains 0 (unchanged).
5.5		Verify writing the SAME (unchanged) div_en/div_val value while running is NOT flagged as an error.	With timer_en=1, div_val=3, div_en=1, issue a write repeating the identical div_val=3, div_en=1 values.	tim_pslverr=0; TCR contents unchanged, as expected.
5.6		Verify one TCR write combining a valid field change with an illegal field change is entirely rejected.	With timer_en=1, issue a single write (pstrb=4'hF) that both sets timer_en=0 (valid) and changes div_val to a different value while the counter is still running (illegal); read back TCR.	tim_pslverr asserted; timer_en is NOT updated either -- the whole transaction is blocked, not only the offending field.
5.7		Verify timer_en itself is always writable with no restriction.	Toggle timer_en 0->1->0 multiple times, independent of div_en/div_val.	Every timer_en write succeeds (tim_pslverr=0); TCR[0] readback always matches the last value written.

Figure 3.2: Vplan2

ID	Item	Description	Test sequence	Pass condition
6.1	apb_protocol_chk	Verify tim_pready timing during a WRITE transfer.	Drive a standard APB write (tim_psel=1, tim_pwrite=1, tim_penable=0 for the setup cycle, then tim_penable=1); sample tim_pready every clock edge from the setup cycle through several cycles after penable asserts.	tim_pready=0 during the SETUP phase (penable=0); tim_pready rises to 1 exactly 1 clock cycle after tim_penable first becomes 1; tim_pready stays high for exactly 1 clock cycle, then returns to 0 and stays 0 for the rest of the transfer (no extra pulses).
6.2		Verify tim_pready timing during a READ transfer.	Drive a standard APB read (tim_psel=1, tim_pwrite=0, tim_penable=0 for the setup cycle, then tim_penable=1); sample tim_pready every clock edge from the setup cycle through several cycles after penable asserts.	tim_pready=0 during the SETUP phase (penable=0); tim_pready rises to 1 exactly 1 clock cycle after tim_penable first becomes 1; tim_pready stays high for exactly 1 clock cycle, then returns to 0 and stays 0 for the rest of the transfer (no extra pulses).
6.3		Verify tim_pready never asserts for a WRITE attempt when the slave is not selected.	Drive tim_pwrite=1 and toggle tim_penable 0->1 exactly as in a normal write, but keep tim_psel=0 for the entire sequence.	tim_pready remains 0 for the entire sequence; the targeted register is not written.
6.4		Verify tim_pready never asserts for a READ attempt when the slave is not selected.	Drive tim_pwrite=0 and toggle tim_penable 0->1 exactly as in a normal read, but keep tim_psel=0 for the entire sequence.	tim_pready remains 0 for the entire sequence; tim_pdata stays at its RAZ/idle value (no valid read occurs).
7.1	apb_multiple_access	Verify back-to-back write transactions with no idle bus cycle between them.	Issue two consecutive write transactions to different addresses (e.g. TCMP0 then TCMP1), keeping tim_psel asserted continuously between them.	Both writes complete correctly with the correct data landing in the correct register; no data corruption or missed transfer.
7.2		Verify a write immediately followed by a read to the same address.	Write a known value to TDR0 (timer_en=0), then immediately issue a read to the same address as the very next transaction.	The read returns exactly the value just written.
7.3		Verify repeated read-modify-write sequences on the same register.	Perform write -> read -> write -> read on TCMP0 with different data each time, back-to-back.	Each read always reflects the immediately preceding write; no stale or corrupted data.
8.1	apb_unaligned_chk	Verify a non-word-aligned address between TCR and TDR0 is treated as reserved.	Access address 0x001, 0x002 and 0x003 with both a write and a read.	Every read returns 32'h0000_0000 (RAZ); the writes have no effect on TCR or TDR0 (WI).
8.2		Verify a non-word-aligned address between TDR0 and TDR1 is treated as reserved.	Access address 0x005, 0x006 and 0x007 with both a write and a read.	Every read returns 32'h0000_0000; the writes have no effect on any live register.
8.3		Verify non-word-aligned addresses in the remaining register gaps are treated as reserved.	Repeat the same write/read pattern at 0x00D-0x00F, 0x011-0x013, 0x015-0x017, 0x019-0x01B and 0x01D-0x01F.	Every read returns 0; no live register is affected by any of the writes.

Figure 3.3: Vplan3

ID	Item	Description	Test sequence	Pass condition
9.1	cnt_counting_chk	Verify undivided counting rate.	Set div_en=0, timer_en=1; sample the 64-bit counter (via TDR0/TDR1) over N clock cycles.	The counter increases by exactly 1 every sys_clk cycle.
9.2		Verify divide-by-1 behavior in control mode.	Set div_en=1, div_val=0, timer_en=1; sample the counter over several cycles.	The counter increases by exactly 1 every sys_clk cycle (same rate as 9.1).
9.3		Verify divide-by-2 ^{div_val} counting rate.	Set div_en=1, 0<div_val<8, timer_en=1; sample the counter over several cycles.	The counter increases by exactly 1 every 2 ^{div_val} sys_clk cycles.
9.4		Verify the maximum divide ratio (divide-by-256).	Set div_en=1, div_val=8, timer_en=1; sample the counter over at least 512 cycles.	The counter increases by exactly 1 every 256 sys_clk cycles.
9.5		Verify the counter keeps counting after an interrupt trigger.	Program TCMP0 so a match occurs shortly after start; let the counter reach and pass the compare value.	The counter keeps incrementing normally after TISR.int_st becomes 1; no stall or reset is caused by the interrupt event.
9.6		Verify 64-bit overflow / wraparound behavior.	Preload TDR0/TDR1 = 64'hFFFF_FFFF_FFFF_FFFF while timer_en=0, then set timer_en=1 and let it tick past the maximum value.	The counter wraps to 64'h0 and continues incrementing normally; no error flag is raised.
9.7		Verify hardware auto-clear when timer_en goes High to-Low.	Let the counter run to a non-zero value, then write timer_en=0 (pwrdata bit0=0) in a single TCR write.	TDR0/TDR1 are synchronously cleared to 0 by hardware at that write, with no separate SW write to TDR0/TDR1.
9.8		Verify counting resumes correctly after re-enabling.	Immediately after 9.7, write timer_en=1 again.	The counter restarts from 0 at the correct rate for the currently configured div_en/div_val.
9.9		Verify a SW-preloaded start value is honored once counting begins.	With timer_en=0, write TDR0/TDR1 to a known non-zero value, then set timer_en=1.	The counter begins incrementing from the preloaded value, not from 0.
10.1	interrupt_chk	Verify TISR.int_st sets on compare match.	Set TCMP0/TCMP1 to a value the counter will reach shortly; run the counter until cnt == (TCMP1, TCMP0).	TISR.int_st (bit0) reads 1 immediately after the matching cycle.
10.2		Verify tim_int reflects (int_en AND int_st).	With int_st=1, toggle TIER.int_en between 0 and 1.	tim_int=0 while int_en=0; tim_int=1 once int_en=1 (int_st still 1).
10.3		Verify SW clear of int_st (RW1C).	With int_st=1, write TISR.bit0=1.	TISR.int_st reads 0 immediately after the write; tim_int deasserts accordingly if it was asserted.
10.4		Verify writing 0 to int_st has no effect.	With int_st=1, write TISR.bit0=0.	TISR.int_st remains 1 (unchanged).
10.5		Verify writing to int_st when it is already 0 has no effect.	With int_st=0, write TISR.bit0=1.	TISR.int_st remains 0 (no spurious set).
10.6		Verify the counter is unaffected by the interrupt condition.	Same setup as 10.1; continue observing the counter for several cycles after the match.	The counter continues incrementing past the match value, with no stall, clear or reload.
10.7		Verify masking int_en does not clear the pending status.	With int_st=1 and int_en=1 (tim_int=1), write TIER.int_en=0.	tim_int deasserts to 0 immediately; TISR.int_st remains 1 (still readable as 1, only the output is masked).
10.8		Verify clear has priority over a simultaneous set.	Arrange for cnt == (TCMP1, TCMP0) (set condition true) on the same clock edge as a SW write of 1 to TISR.bit0 (int_st currently 1).	int_st reads 0 immediately after that edge (clear wins over set).

Figure 3.4: Vplan4

ID	Item	Description	Test sequence	Pass condition
11.1	cnt_halt_chk	Verify the halt_ack truth table.	Sweep dbg_mode and THCSR halt_req through all 4 combinations (00, 01, 10, 11); read THCSR bit1 each time.	halt_ack = dbg_mode AND halt_req for all 4 combinations (1 only when both inputs are 1).
11.2		Verify the counter freezes when halted.	With timer_en=1 and the counter running, set dbg_mode=1 and halt_req=1; sample the counter over several clock cycles while halted.	The counter value stays constant (no increments) for the entire duration halt_ack=1.
11.3		Verify the halt does NOT engage with only one of the two conditions true.	Case A: halt_req=1, dbg_mode=0. Case B: halt_req=0, dbg_mode=1. Run the counter under each case.	The counter keeps incrementing normally at its configured rate in both Case A and Case B (halt_ack=0 in both).
11.4		Verify the internal tick phase is preserved across a halt (div_val>0).	Set div_en=1, div_val=2 (divide-by-4); halt in the middle of a divide window (e.g. after 2 of 4 sub-cycles); hold the halt for an arbitrary number of cycles, then release it.	The total sys_clk cycles actually spent counting for that specific tick equals exactly 4 (the configured divide ratio); only the 2 remaining sub-cycles are needed after resume, matching the design-spec waveform example.
11.5		Verify counting resumes correctly after halt is released.	After 11.2, clear halt_req back to 0 (dbg_mode may stay 1 or go to 0).	The counter resumes incrementing at the correct rate from exactly the value it was frozen at.
11.6		Verify halting exactly at a compare-match value does not block interrupt detection.	Arrange for the halt to engage exactly when cnt == {TCMP1, TCMP0}.	TISR.int_st still becomes 1 at the matching edge; the counter remains frozen at that value until the halt is released.
12.1	apb_pslverr_chk	Verify writing div_en while the counter is stopped never causes an error.	With timer_en=0, write TCR div_en=0; read back and check pslverr. Then write TCR div_en=1; read back and check pslverr.	tim_pslverr=0 for both writes (div_en_err requires timer_en=1, which is false here); div_en_g follows each write.
12.2		Verify the div_val range check fires purely on the prohibited-value rule, independent of timer_en.	With timer_en=0, sweep TCR div_val across the full 4-bit range 0 to 15, one write per value; check pslverr after each write.	pslverr=0 for div_val=0..8; pslverr=1 for div_val=9..15 on every write in that range, even though timer_en=0.
12.3		Verify changing div_en while running is an error only when the value actually changes, starting from div_en=0.	Set timer_en=1 with div_en=0. Write div_en=0 (no change); check pslverr. Then write div_en=1 (change); check pslverr.	pslverr=0 for the div_en=0 write; pslverr=1 for the div_en=1 write; div_en_g stays 0 after the erroring write.
12.4		Verify changing div_en while running is an error only when the value actually changes, starting from div_en=1.	Set timer_en=1 with div_en=1. Write div_en=1 (no change); check pslverr. Then write div_en=0 (change); check pslverr.	pslverr=0 for the div_en=1 write; pslverr=1 for the div_en=0 write; div_en_g stays 1 after the erroring write.
12.5		Verify changing div_val while running is an error for any value other than the currently-held value, across the full 0-15 range.	Set timer_en=1 with a known div_val (e.g. 3). With timer_en held at 1, sweep TCR div_val writes across 0 to 15 one at a time; check pslverr after each write.	pslverr=0 only for the write equal to the currently-held div_val (3); pslverr=1 for every other value 0-15 (0-8 via the changed-while-running rule, 9-15 via that rule and the prohibited-value rule together); div_val_g never actually changes from 3 throughout the sweep.

Figure 3.5: Vplan5

3.3 Testbench Organization

The self-checking testbench implementing the strategy of Section 3.2 is organized under `tb/` in the project repository; the full source of every file below is available there. This section summarizes its structure.

3.3.1 Top-level Testbench and Golden Reference Model

`test_bench` instantiates the DUT (`timer_top`) side-by-side with a cycle-accurate `timer_golden_model`, generates the 20 MHz system clock (50 ns period), and drives the shared APB interface signals into both.

`timer_golden_model` maintains shadow copies of every register (`timer_en_g`, `div_en_g`, `div_val_g`, `cnt_g`, `tcmp0_g`/`tcmp1_g`, `int_en_g`, `int_st_g`, `halt_req_g`) and re-implements the prescaler, compare-match, halt, and RW1C interrupt logic in parallel with the DUT to produce the expected values `prdata_exp`, `pslverr_exp`, and `tim_int_exp` used for scoreboarding.

3.3.2 Bus Driver Tasks

A set of reusable Verilog tasks drive and check APB transactions:

- **reset** — applies a full register reset sequence, bringing the DUT back to its default configuration between test scenarios.

- `wr_task` / `rd_task` — drive a single APB write or read transfer following the 1-cycle wait-state handshake; `rd_task` automatically compares `tim_prdata` against `prdata_exp` and reports a [PASSED]/[FAILED] verdict.
- `wr_task_err_chk` — drives an APB write and checks `tim_pslverr` against `pslverr_exp`, used for illegal-configuration test scenarios.
- `rd_task_out` — performs an APB read and returns the read data to the caller, used to seed subsequent test steps with the current register value.

3.3.3 Directed Test Sequences

Twelve directed test blocks exercise the Verification Plan of Section 3.2, each ending with a self-checked PASSED banner:

- `REG_INT_CHK` — verifies the reset values of all eight registers.
- `REG_RW_CHK` — verifies full read/write access to `TDR0/1`, `TCMP0/1`, `TCR` (`div_val` sweep from 0 to 8), `TIER`, and `THCSR`.
- `REG_RESERVED_CHK` — verifies RAZ/WI behavior of unmapped addresses and the reserved bit-fields of `TCR`, `TIER`, `TISR`, and `THCSR`.
- `REG_BYTE_ACCESS_CHK` — verifies byte-strobe write isolation on `TDR0` and `TCMP0`, including a non-contiguous strobe pattern and a zero-strobe write.
- `CNT_COUNTING_CHK` — verifies undivided and divided counting rates, continued counting across a compare match, 64-bit overflow wraparound, hardware clear on `timer_en` $H \rightarrow L$, and resumption from a software-preloaded value.
- `INTERRUPT_CHK` — verifies `int_st` assertion on match, `RW1C` clearing, write-0 no-effect, dynamic `int_en` masking, and clear-vs-set priority.
- `APB_PSLVERR_CHECK` — sweeps `div_val` and toggles `div_en`/`timer_en` to verify `tim_pslverr` generation for every illegal configuration write.
- `CNT_CTRL_CHECK` — verifies configuration-write protection while `timer_en` = 1, including legal no-op writes and combined legal/illegal writes in the same transfer.
- `APB_MULTIPLE_ACCESS_CHECK` — verifies back-to-back write transfers and write-immediately-followed-by-read sequences.
- `APB_UNALIGNED_CHECK` — sweeps every non-word-aligned address in the register range and verifies RAZ/WI behavior.

- CNT_HALT_CHECK — sweeps dbg_mode/halt_req combinations, verifies counter freeze/resume and prescaler-phase preservation, including a compare match occurring while halted.
- APB_PROTOCOL_CHECK — verifies tim_pready timing on write and read transfers, and confirms that unselected (tim_psel = 0) accesses are ignored.

3.4 Simulation Results

3.4.1 Test Results Summary

```
# -----
# -----
# -----
# ----- REG_INT_CHK -- PASSED -----
# ----- REG_RW_CHK -- PASSED -----
# ----- REG_RESERVED_CHK -- PASSED -----
# -- REG_BYTE_ACCESS_CHK -- PASSED --
# ----- CNT_CTRL_CHECK -- PASSED -----
# --- APB_PROTOCOL_CHECK -- PASSED ---
# --APB_MULTIPLE_ACCESS_CHECK -- PASSED--
# -- APB_UNALIGNED_CHECK -- PASSED --
# ----- CNT_COUNTING_CHK -- PASSED -----
# ----- INTERRUPT_CHK -- PASSED -----
# ----- CNT_HALT_CHECK -- PASSED -----
# -- APB_PSLVERR_CHECK -- PASSED --
# -----
# -----
# -----
```

Figure 3.6: result

3.4.2 Coverage Results

```

=====
=== Instance: /test_bench/dut/apb_slave_u
=== Design Unit: work.apb_slave
=====

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	4	4	0	100.00%
Expressions	26	26	0	100.00%
Statements	9	9	0	100.00%
Toggles	2	2	0	100.00%

```

=====
=== Instance: /test_bench/dut/register_u
=== Design Unit: work.register
=====

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	51	51	0	100.00%
Conditions	6	6	0	100.00%
Expressions	44	44	0	100.00%
Statements	62	62	0	100.00%
Toggles	366	366	0	100.00%

```

=====
=== Instance: /test_bench/dut/cnt_ctrl_u
=== Design Unit: work.cnt_ctrl
=====

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	19	19	0	100.00%
Expressions	7	7	0	100.00%
Statements	20	20	0	100.00%
Toggles	36	36	0	100.00%

Figure 3.7: Coverage1

```

=====
=== Instance: /test_bench/dut/cnt_u
=== Design Unit: work.cnt
=====

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	22	22	0	100.00%
Expressions	16	16	0	100.00%
Statements	20	20	0	100.00%
Toggles	144	144	0	100.00%

```

=====
=== Instance: /test_bench/dut/interrupt_u
=== Design Unit: work.interrupt
=====

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	6	6	0	100.00%
Expressions	2	2	0	100.00%
Statements	6	6	0	100.00%
Toggles	2	2	0	100.00%

```

=====
=== Instance: /test_bench/dut
=== Design Unit: work.timer_top
=====

```

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	170	170	0	100.00%

Figure 3.8: Coverage2

Appendix A

Additional Material

A.1 Source Listings

The complete RTL source (`rtl/`), testbench source and golden reference model (`tb/`), and the Verification Plan (VPlan) matrix and RTL logic diagrams (`doc/`) referenced throughout this report are included directly in the project repository alongside this document.